Work Management

Work management is the layer that turns a stream of authoring intent into bounded, observable labor. In the Codynamic Book Machine, the unit of scheduling is the *WorkItem*: a discrete task with a clear purpose, an owner or target agent, a status, an estimate of effort, and enough context to resume work without reconstructing the entire conversation. WorkItems are used for drafting, review, diagram requests, reference lookup, cleanup, and assembly tasks, but they all share the same operational shape: they can be queued, blocked, checkpointed, and completed.

A *WorkManager* coordinates these items across the system. Its role is not to perform the work itself, but to maintain the queue discipline that keeps the machine coherent. It accepts new items, orders them by priority and dependency, tracks which items are active, and records when an item is waiting on another item, a missing artifact, or a human decision. In this sense, the WorkManager is the operational counterpart to the canonical work object: the work object describes what the book is, while the WorkManager describes what must happen next.

A useful way to understand this layer is to follow a WorkItem through its lifecycle. An item may be created from an outline requirement, a revision request, or a coordination need. It enters the queue with a priority and a dependency set, becomes active when capacity is available, and either advances to completion or pauses in a blocked state if it cannot proceed. At each meaningful boundary, the item can be checkpointed so that its current state is preserved without forcing the agent to repeat earlier reasoning.

Checkpoints are the mechanism that makes long-running work resumable. A checkpoint captures the current state of a WorkItem at a meaningful boundary: what has been completed, what remains, what assumptions were made, and what external inputs are still pending. For section drafting, a checkpoint may include the current LaTeX body, unresolved citation needs, open diagram requests, and any editorial constraints inherited from the outline. For coordination tasks, a checkpoint may record the last known message state or the latest decision from the hypervisor. Checkpoints reduce the cost of interruption and make it possible to hand work between agents without losing continuity.

Capacity reports provide the system with a simple but important form of self-observation. A capacity report summarizes how much work an agent or queue can reasonably accept at a given moment, how many items are active, what kinds of tasks are consuming attention, and whether the current load is sustainable. This is especially important in a multi-agent environment, where the appearance of parallelism can hide a practical bottleneck. If one agent is overloaded with high-priority items, the report should make that visible before the queue becomes unstable.

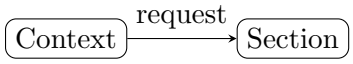
The WorkManager also tracks blockers explicitly. A blocker is any condition that prevents a WorkItem from advancing: a missing reference key, an unresolved dependency, a required diagram that has not yet been generated, a conflicting instruction, or a downstream artifact that has not been registered. Blockers are not treated as vague delays; they are first-class state. That distinction matters because it allows the system to distinguish between work that is merely waiting and work that is structurally impossible to complete until another action occurs.

Overcommitment is the failure mode that queue discipline is designed to prevent. When too many items are accepted at once, the system may appear productive while actually accumulating hidden debt: partially drafted sections, unresolved citations, duplicated requests, and inconsistent revisions. The WorkManager therefore needs to enforce limits on concurrent work and to surface when the queue exceeds available capacity. Overcommitment is not only a scheduling problem; it is a quality problem, because rushed coordination tends to produce artifacts that are harder to verify and revise.

Queue discipline is the rule set that keeps this from happening. Items should enter the queue

with explicit priority and dependency information, should move forward only when their prerequisites are satisfied, and should be marked blocked rather than silently stalled. The queue should prefer small, finishable units of work over broad, ambiguous ones, because smaller items are easier to checkpoint, easier to review, and easier to reassign. In practice, this means the system favors a steady flow of completed WorkItems over a large backlog of partially addressed ones.

Taken together, WorkItem, WorkManager, checkpoints, capacity reports, blockers, overcommitment, and queue discipline define the operational grammar of the machine. They ensure that authoring is not just a sequence of text generations, but a managed process in which progress is visible, interruptions are survivable, and coordination remains bounded by explicit state. In the next section, that operational grammar extends into the message layer, where routed prompts, durable logs, and runtime memory make the coordination records themselves persistent.



Conceptual diagram for 'Work Management': Explain WorkItem, WorkManager, checkpoints, capacity reports, blockers, overcommitment, and queue discipline.

Figure 1: Conceptual diagram for 'Work Management': Explain WorkItem, WorkManager, checkpoints, capacity reports, blockers, overcommitment, and queue discipline.